==========

RISE — AI ALARM CLOCK

MASTER BLUEPRINT: PART 5 OF 13 (REWRITTEN — CHANGESET)

Sleep Detection — Wearable-First, Accelerometer Fallback

==========

GOVERNING DOCUMENT: RISE_DECISIONS.md Section 3.2.

WHAT THIS REWRITE FOUND: the original Part 5's own UI copy already

said "Sleep estimated from screen-off time and motion. For greater

accuracy, connect a wearable" — a promise with no mechanism behind

it anywhere in the original 13 parts. This rewrite is what actually

builds that promised connection, not a new idea layered on top.

WHAT DID NOT CHANGE, DELIBERATELY: the EC-4.10 "always label as

estimated" discipline in the original Part 5 is genuinely well

built and is NOT being loosened — it's being joined by a second,

more confident path for users with a connected wearable, while the

accelerometer path keeps its honest "estimated" labeling exactly as

designed. The real-time state machine (pre-sleep timer, micro-waking

detection, fall detection, travel mode, pillow advisory) is

UNCHANGED — those are alarm-adjacent behaviors that matter in the

moment regardless of whether a wearable is connected, so

sleep_detection_service.dart needs no changes at all. The wearable

integration happens entirely at the provider layer, above it.

FILES IN THIS PART:

1. motion_analyzer.dart — NO CHANGE

2. sleep_detection_service.dart — NO CHANGE (see note above)

3. thermal_monitor.dart — NO CHANGE

4. sleep_quality_scorer.dart — CHANGED (wearable-sourced path added)

5. sleep_detection_provider.dart — CHANGED (wearable check added)

6. sleep_tracker_screen.dart — CHANGED (honest source labeling)

NEW: lib/core/services/wearable_data_bridge.dart

==========

FILE 1: lib/core/services/motion_analyzer.dart

NO CHANGE. The raw accelerometer signal processing is exactly as

valuable as before — it's now explicitly the FALLBACK engine

instead of the only engine, but the algorithm itself is untouched.

FILE 2: lib/core/services/sleep_detection_service.dart

NO CHANGE. The state machine (awake → preSleep → sleeping →

microWaking) continues to drive real-time, alarm-adjacent behavior

exactly as designed — EC-4.1 through EC-4.7 all still apply

unmodified. Wearable data does not replace this; it supplements the

FINAL SCORED RECORD shown to the user, handled entirely at the

provider layer (FILE 5, below).

FILE 3: lib/core/services/thermal_monitor.dart

NO CHANGE. Unrelated to the wearable-first decision.

NEW FILE: lib/core/services/wearable_data_bridge.dart

PURPOSE: Wraps the `health` package (Apple HealthKit + Google

Health Connect) to fetch last night's sleep session

directly from a connected wearable when one exists.

NOTE: the shape below reflects the `health` package's general API

as of this rewrite (Health() class, HealthDataType enum,

requestAuthorization). VERIFY exact method signatures against

the installed version's current docs before implementing —

this package has had breaking API changes across major

versions, more so than most Flutter packages.

```
import 'package:health/health.dart';

enum WearableSource { none, appleHealth, healthConnect }

class WearableSleepData {
  final WearableSource source;
  final String         deviceLabel;   // e.g. "Apple Watch", "Oura Ring"
  final DateTime        sleepStart;
  final DateTime        sleepEnd;
  final int             remMinutes;
  final int             deepMinutes;
  final int             lightMinutes;
  final int             awakeMinutes;
  final int?            avgHeartRateBpm;

  const WearableSleepData({
    required this.source,
    required this.deviceLabel,
    required this.sleepStart,
    required this.sleepEnd,
    required this.remMinutes,
    required this.deepMinutes,
    required this.lightMinutes,
    required this.awakeMinutes,
    this.avgHeartRateBpm,
  });
```

```
  int get totalMinutes => remMinutes + deepMinutes + lightMinutes;
}

class WearableDataBridge {
  static final WearableDataBridge _instance =
WearableDataBridge._internal();
  factory WearableDataBridge() => _instance;
  WearableDataBridge._internal();

  final Health _health = Health();

  static const _sleepTypes = [
    HealthDataType.SLEEP_ASLEEP,
    HealthDataType.SLEEP_AWAKE,
    HealthDataType.SLEEP_DEEP,
    HealthDataType.SLEEP_REM,
    HealthDataType.SLEEP_LIGHT,
    HealthDataType.HEART_RATE,
  ];

  Future<bool> requestPermissions() async {
    return await _health.requestAuthorization(_sleepTypes);
  }

  // A wearable is "connected" for RISE's purposes only if permission
  // is granted AND real sleep samples exist in the last 7 days —
  // granted permission alone doesn't mean the user actually wears a
  // device to bed, and showing "connected" when there's no data would
  // be its own honesty violation (same principle as EC-4.10).
  Future<bool> isConnected() async {
    final hasPermission = await _health.hasPermissions(_sleepTypes) ??
false;
    if (!hasPermission) return false;

    final recent = await _health.getHealthDataFromTypes(
      startTime: DateTime.now().subtract(const Duration(days: 7)),
      endTime:   DateTime.now(),
      types:     [HealthDataType.SLEEP_ASLEEP],
    );
    return recent.isNotEmpty;
  }

  // Returns null if no wearable sleep data exists for last night
  // (device not worn, not synced yet, or not connected at all) — the
  // caller falls back to the accelerometer path in that case.
  Future<WearableSleepData?> fetchLastNight() async {
    final now  = DateTime.now();
    final from = now.subtract(const Duration(hours: 20));

    final samples = await _health.getHealthDataFromTypes(
      startTime: from,
```

```
      endTime:   now,
      types:     _sleepTypes,
    );

    if (samples.isEmpty) return null;

    // Aggregate stage minutes from the raw samples. Exact aggregation
    // logic (merging overlapping intervals, handling multi-source data
    // if both Apple Watch and a ring are connected) needs full
    // implementation against the real sample shape — sketched here at
    // the interface level so the provider layer (FILE 5) has a stable
    // contract to build against.
    return _aggregateToNightlySummary(samples);
  }

  WearableSleepData _aggregateToNightlySummary(List<HealthDataPoint>
samples) {
    // Implementation detail — bucket samples by type, sum durations,
    // determine deviceLabel from the platform's source metadata.
    throw UnimplementedError('Aggregation logic — implement against real
HealthDataPoint shape');
  }
}
```

FILE 4: lib/core/services/sleep_quality_scorer.dart — CHANGED

Adds a wearable-sourced construction path alongside the existing

accelerometer-based scoring. The EC-4.10 estimated-labeling fields

are UNCHANGED for the accelerometer path; two new fields added to

the output class so the UI (FILE 6) can tell the two apart.

```
class ScoredSleepSession {
  // ─── Existing fields — UNCHANGED ──────────────────────────────────
```

```
  final int    estimatedRemMinutes;
  final int    estimatedDeepMinutes;
  final int    estimatedLightMinutes;
  // ...(all other original fields unchanged, not reproduced)

  // ─── NEW
──────────────────────────────────────────────────────────────────────────────
──────────────────────

  final bool    isFromWearable;
  final String? wearableDeviceLabel;  // null when accelerometer-based

  const ScoredSleepSession({
    required this.estimatedRemMinutes,
    required this.estimatedDeepMinutes,
    required this.estimatedLightMinutes,
    this.isFromWearable      = false,
    this.wearableDeviceLabel,
    // ...other required original fields
  });

  // NEW: construct directly from wearable data — no heuristic
  // estimation needed, the wearable already measured this.
  factory ScoredSleepSession.fromWearable(WearableSleepData data) {
    return ScoredSleepSession(
      estimatedRemMinutes:   data.remMinutes,
      estimatedDeepMinutes:  data.deepMinutes,
      estimatedLightMinutes: data.lightMinutes,
      isFromWearable:        true,
      wearableDeviceLabel:   data.deviceLabel,
      // ...other fields populated from `data` accordingly
    );
  }

  // The existing accelerometer-based scoring method (score(), and
  // whatever heuristic math produces the REM/deep/light split from
  // motion + duration) is UNCHANGED — still sets isFromWearable:
  // false implicitly via the default above, still carries the
  // EC-4.10 estimated framing exactly as before.
}
```

FILE 5: lib/core/providers/sleep_detection_provider.dart — CHANGED

This is where the wearable-first decision actually happens. Two

new providers added; lastNightScoredProvider changed to check the

wearable bridge before falling back to the original accelerometer

path. All other providers (sleepStateProvider, sleepTrendProvider,

avgSleepDurationProvider, lastNightSessionProvider,

sleepDetectionServiceProvider) are UNCHANGED.

```
// ── NEW ──────────────────────────────────────────────────────────────────────
final wearableBridgeProvider = Provider<WearableDataBridge>(
  (_) => WearableDataBridge(),
);

final wearableConnectedProvider = FutureProvider<bool>((ref) async {
  return ref.watch(wearableBridgeProvider).isConnected();
});

// ── CHANGED: wearable-first, accelerometer fallback ──────────────────────────
final lastNightScoredProvider = FutureProvider<ScoredSleepSession?>
((ref) async {
  final bridge       = ref.watch(wearableBridgeProvider);
  final wearableData = await bridge.fetchLastNight();

  if (wearableData != null) {
    return ScoredSleepSession.fromWearable(wearableData);
  }

  // No wearable data for last night — fall back to the original,
  // fully unchanged accelerometer-based path.
  final session = ref.watch(lastNightSessionProvider);
  if (session == null) return null;
  return SleepQualityScorer().score(session);  // original call,
unchanged
});
```

FILE 6: lib/features/sleep/sleep_tracker_screen.dart — CHANGED

Only the honesty-label section changes. Everything else (pillow

advisory, travel-mode banner, live state card, session card, 7-day

average, trend chart, recent sessions list) is UNCHANGED.

```
// ─── REPLACES the original static "estimated" Text widget
──────────────────
final wearableConnected = ref.watch(wearableConnectedProvider);

wearableConnected.when(
  data: (connected) => Text(
    connected
        ? 'Sleep data from your connected device.'
        : 'Sleep estimated from screen-off time and motion. '
          'For greater accuracy, connect a wearable.',
    style: TextStyle(color: Colors.white.withOpacity(0.45), fontSize:
12),
  ),
  loading: () => const SizedBox.shrink(),
  // On error, fail toward the original honest-by-default copy rather
  // than silently showing nothing — same EC-4.10 spirit.
  error: (_, __) => Text(
    'Sleep estimated from screen-off time and motion. '
    'For greater accuracy, connect a wearable.',
    style: TextStyle(color: Colors.white.withOpacity(0.45), fontSize:
12),
  ),
),

// A tappable "Connect a wearable" affordance when `connected` is
// false belongs here too — routes to a settings/permission flow.
// That flow's screen is a Part 11 concern (onboarding/settings), not
// specified further in this Part.
```

PART 5 FILE INVENTORY (rewritten)

NEW: wearable_data_bridge.dart
CHANGED: sleep_quality_scorer.dart, sleep_detection_provider.dart, sleep_tracker_screen.dart
UNCHANGED: motion_analyzer.dart, sleep_detection_service.dart, thermal_monitor.dart

OPEN ITEM FOUND DURING THIS REWRITE

`_aggregateToNightlySummary()` in wearable_data_bridge.dart is sketched at the interface level, not fully implemented — turning raw HealthDataPoint samples into a single nightly summary (merging overlapping intervals, picking a device label when multiple sources report data) needs to be built against the real sample shape once the `health` package version is pinned. Flagging this now rather than guessing at implementation details I haven't verified.

WHAT'S NEXT (per DECISIONS.md Section 9)

Part 8 / 10B — meal signal tracker replaces the full calorie tracker.